

41. Animer les couleurs : l'eau qui scintille

Faire tourner trois couleurs d'une palette, et toute une mer ondule — sans redessiner une tuile.

1. Ce qu'il faut comprendre

Toutes les cases teintées en palette 1 changent **ensemble** quand on change la palette 1. C'est la ruse de l'**animation de palette** : une mer de 60 cases ondule, et l'on n'a écrit que **trois couleurs**.

Les trois bleus sont rangés dans trois tables, **ROUGES**, **VERTS** et **BLEUS** — pas **R**, **V**, **B** : **B** est déjà le nom du bouton B, et le compilateur le refuse. Tous les 10 tours, **decalage** avance d'un cran, et chaque indice prend la couleur **suivante** : $(i + \text{decalage}) \% 3$. Le modulo fait tourner la liste en rond — le bleu clair passe à l'indice 2, le moyen à l'indice 3, le sombre revient à l'indice 1.

Redessiner 60 tuiles demanderait 60 écritures, chacune attendant sa pause (chapitre 3), et cela à chaque image de l'animation. Changer trois couleurs n'en coûte presque rien. C'est ainsi que les jeux font scintiller l'eau, luire la lave, clignoter un néon ou tomber la nuit.

Même idée qu'au fondu du chapitre 7 : **on ne touche pas au dessin, on change sa lecture**.

À toi : fais un coucher de soleil — le ciel qui passe lentement du bleu à l'orange, puis au violet.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **720 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	49 o	← la console, à l'allumage — la mise en route de la console
main()	288 o	← démarrage — ta fonction
VBlank	8 o	← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage	28 o	← image() — attend l'image suivante
AttendreVBlank	16 o	← ecran() — attend que l'écran soit libre
AttendreAcces	12 o	← EcrireTexte — attend de pouvoir écrire une case
AdresseCase	10 o	← poser(), teindre() — trouve l'adresse d'une case de l'écran
EcrireTexte	10 o	← texte() — écrit un texte, lettre par lettre
Reste	10 o	← main() — le reste d'une division
CopierTuiles	94 o	← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage — vide les deux cartes de l'écran
Tuiles	160 o	← CopierTuiles — 9 lettres sur 43, et 1 tuile dessinée
"LA MER BOUGE"	12 o	← texte() — un texte gravé
ROUGES_0	3 o	← couleurFond() — une table gravée
VERTS_1	3 o	← couleurFond() — une table gravée
BLEUS_2	3 o	← couleurFond() — une table gravée

Lettres gravées : ABEGLMORU

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EffacerCases, EcrireNombre, Diviser, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

couleurFond()	× 2	106 o sur place, dans main() + ROUGES_0 (3 o), VERTS_1 (3 o), BLEUS_2 (3 o)
teindre()	× 1	24 o sur place, dans main() + AdresseCase (10 o)
ecran()	× 2	21 o sur place, dans main() + AttendreVBlank (16 o)
poser()	× 1	13 o sur place, dans main() + AdresseCase (10 o)

```
texte()      × 1    11 o sur place, dans main() + EcrireTexte (10 o)
image()      × 1    3 o sur place, dans main() + AttendreImage (28 o)
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// CE PROGRAMME (Game Boy Color) : une mer de vagues qui scintille.
//
// L'astuce (« animation de palette ») : on ne redessine AUCUNE tuile.
// On fait tourner trois bleus entre les indices 1, 2 et 3 de la palette 1 :
// toutes les cases teintées en palette 1 changent ensemble.

#include <reste>    // a % b, sauf par 1, 2, 4, 8, 16... écrits en clair

#include <Tuile>      // un dessin de 8 × 8 pixels
#include <couleurFond> // choisit une couleur d'une palette du fond
#include <ecran>      // éteint ou rallume l'écran
#include <texte>       // écrit un texte à l'écran
#include <poser>       // pose une tuile sur une case du fond
#include <teindre>     // met une case du fond dans une palette
```

2. UN DESSIN — VAGUE

```
Tuile VAGUE = {
    // Des bandes en diagonale d'indices 1, 2, 3.
    "11112222", "11222233", "12223333", "22233331",
    "22333311", "23333111", "33331112", "33311122",
};

// Trois bleus, du clair au sombre. Chaque table donne une lumière :
//      bleu 0  bleu 1  bleu 2
```



à l'écran

3. UNE TABLE GRAVÉE DANS LA CARTOUCHE — ROUGES

```
const uint8_t ROUGES[] = { 8, 2, 0 };
```

4. UNE TABLE GRAVÉE DANS LA CARTOUCHE — VERTS

```
const uint8_t VERTS[] = { 24, 14, 6 };
```

5. UNE TABLE GRAVÉE DANS LA CARTOUCHE — BLEUS

```
const uint8_t BLEUS[] = { 31, 28, 18 };
// (Pas R, V, B : B est déjà le nom du bouton B, le compilateur refuserait.)
```

6. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {                                // Le jeu commence ici.
    couleurFond(1, 0, 31, 31, 31);          // Palette 1, indice 0 : blanc.

    ecran(0);                               // Dessin écran éteint.
    texte(3, 2, "LA MER BOUGE");
    for (uint8_t l = 6; l < 12; l++) {      // Lignes 6 à 11,
        for (uint8_t c = 0; c < 20; c++) {  // toute la largeur :
            poser(c, l, VAGUE);             // une vague,
            teindre(c, l, 1);               // en palette 1. 6 × 20 = 120 cases.
        }
    }
    ecran(1);                               // On rallume.

    uint8_t decalage = 0;                   // De combien on fait tourner les bleus : 0, 1 ou 2.
    uint8_t attente = 0;                   // Compte les images jusqu'à 10.

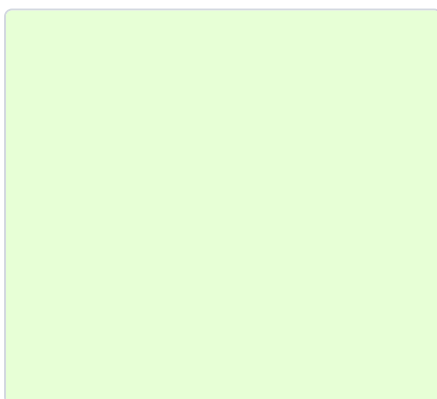
    while (true) {                          // La boucle du jeu :
        image();

        attente++;
        if (attente == 10) {               // Toutes les 10 images :
            attente = 0;
            decalage = (decalage + 1) % 3;  // 0 → 1 → 2 → 0 → 1 ... (le reste par 3 tourne en rond)
        }

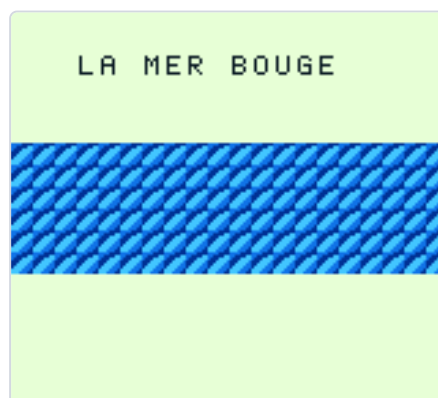
        // Les teintes 1, 2, 3 prennent les trois bleus, décalés.
        for (uint8_t i = 0; i < 3; i++) {  // i = 0, 1, 2
            uint8_t k = (i + decalage) % 3; // quel bleu pour cet indice.
            // decalage = 1 : i = 0 → k = 1 ; i = 1 → 2 ; i = 2 → 0.
            couleurFond(1, i + 1, ROUGES[k], VERTS[k], BLEUS[k]); // l'indice i + 1 (1, 2, 3)
                                                                    // reçoit le bleu numéro k.
        }
    }
}
```

3. Ce qui se passe, étape par étape

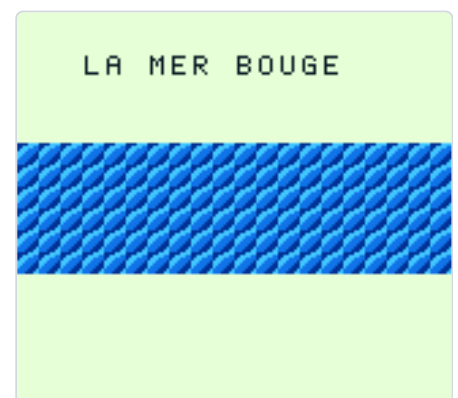
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60

4. Ce qu'on doit voir

Une mer de vagues bleues qui scintille : les bleus tournent, les tuiles ne bougent pas.

5. Ce que la console répond

Ces contrôles sont ceux que `verification/verifier-cours.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ le même pixel passe par les trois bleus (2/14/28 0/6/18 8/24/31)

✓ ce sont bien les bleus de la table

✓ la tuile, elle, n'a pas changé

← [40. Les lutins en couleur](#)

[sommaire](#)

[42. Un jeu lisible en couleur comme en nuances](#) →
